

| BIOSTADÍSTICA

**Universidad Nacional de Jujuy | Facultad de Ciencias Agrarias | Lic. en Ciencias
Biológicas**

Juan Manuel Solís

2026-08-11

Tabla de contenidos

	4
1 Pautas de cursada	5
1.1 Modalidad: Aprendizaje Basado en Competencias	5
2 Competencias prácticas de la bioestadística	7
2.1 Niveles de competencias	7
2.2 Evaluación práctica	8
2.3 Estrategia pedagógica	9
2.4 Estrategias de estudio	10
2.5 Inteligencia artificial generativa: Pautas de trabajo en el curso	11
2.5.1 ¿Qué puede aportar la IA en Bioestadística?	12
2.5.2 Lo que la IA NO puede hacer	12
2.5.3 Esquema de trabajo propuesto	12
3 Introducción al manejo de R	14
3.1 Primeros pasos	14
3.1.1 Otros IDE	16
3.2 Objetos	17
3.3 Funciones	21
3.3.1 Paquetes	22
3.3.2 Concatenación de funciones: el operador “ >” (<i>pipe</i>)	23
3.4 Estructuras de datos	23
3.4.1 El vector	23
3.4.2 Matrices	26
3.4.3 Uniones	26
3.4.4 Data frames	27
3.4.5 Otras estructuras	28
3.5 Importación de datos	28
3.5.1 Archivos .xlsx	28
3.5.2 Archivos .csv	28
3.5.3 Archivos en general (librería “{rio}”)	29
3.5.4 Desde el repositorio de R	29
3.6 Exploración de datos	29
3.7 Exportación de datos	30

3.8	Funciones (Profundización, lectura optativa)	30
3.8.1	Estructura general de una función	30
3.8.2	Estructuras de control	32
3.9	Micro-ejercicios de aplicación (optativos)	34
4	Algunas herramientas	40
	R en la web	40
4.1	GeoGebra (Visualización interactiva)	40
	Referencias	42

LICENCIATURA EN **CIENCIAS BIOLÓGICAS**

MATERIA

BIOESTADÍSTICA



Facultad
de Ciencias
Agrarias



UNJu
Universidad
Nacional de Jujuy

Atención: esta guía se irá actualizando paulatinamente.

1 Pautas de cursada

1.1. Modalidad: Aprendizaje Basado en Competencias

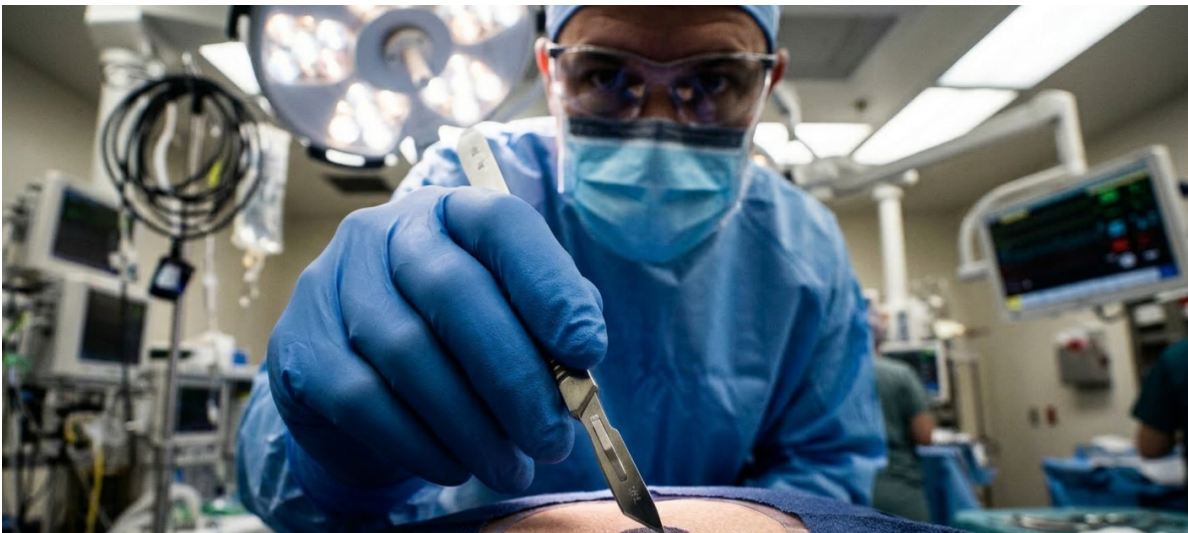
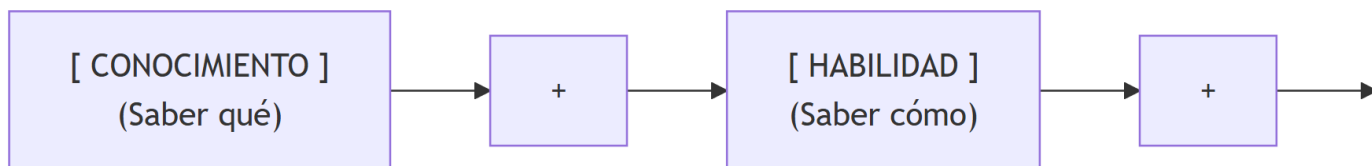


Figura 1.1: Saber, saber hacer, saber ser



Modelo de competencias en Bioestadística

i Descripción de la cursada

- La modalidad del dictado presencial es de tipo taller o laboratorio de análisis estadístico, donde los alumnos deben aplicar los conceptos desarrollados durante la introducción teórica en la resolución de ejercicios propuestos utilizando software estadístico. A tal fin, se admite la conformación de grupos de

hasta dos estudiantes por computadora. Los estudiantes tendrán libertad de asistir a cualquiera de las comisiones prácticas, siempre y cuando hayan asistido previamente a la clase teórica del tema en curso, debido a que la cátedra adoptará una modalidad teórico-práctica.

- Las actividades se complementan con al menos una actividad de campo, para la obtención de datos y su posterior análisis y aplicación de los métodos aprendidos.
- También se implementa un trabajo o Proyecto de Investigación, en el cual los estudiantes deben analizar datos proporcionados en el contexto de casos reales o publicaciones realizadas y comparar sus conclusiones y el planteo de resolución que propusieron, con los descritos en las publicaciones. O bien, la aplicación de análisis de datos en estudios que hayan realizado previamente para otras materias.
- Existe una instancia de evaluación permanente por medio de ejercicios de entrenamiento y auto-evaluación proporcionados en el Aula Virtual, que los alumnos deben resolver fuera del horario de clase. Estos ejercicios estarán acompañados por mecanismos de retroalimentación en la forma de encuestas y monitoreo para realizar el seguimiento de sus actividades. Deberán tener aprobadas todas las actividades de entrenamiento para poder acceder a los parciales correspondientes.
- La nota obtenida en cada parcial se compone de un promedio entre la nota del parcial y de las actividades prácticas.

2 Competencias prácticas de la bioestadística

2.1. Niveles de competencias

Se adopta la jerarquía de niveles de conocimiento estadístico propuesta por **Ben-Zvi y Garfield (2005)**: a) alfabetización estadística, b) razonamiento estadístico y c) pensamiento estadístico, como un continuo de desarrollo formativo, adaptado a un entorno de múltiples herramientas informáticas. Desde una perspectiva práctica, esto significa que cada nivel no solo se declara, sino que se pone en juego en la resolución concreta de los ejercicios propuestos. El motivo es que el profesional no sólo debe reconocer qué procedimiento estadístico es adecuado para cada situación, sino que también debe saber cómo implementarlo.

Cada nivel de conocimiento estadístico se construye sobre el anterior, incorporando progresivamente elementos de niveles superiores a lo largo de la cursada.

- Alfabetización estadística: representa la comprensión del “qué” (conceptos, gráficos, símbolos, funciones). En la práctica, supone reconocer el procedimiento o sintaxis necesarios para ejecutar una prueba, leer correctamente su salida y verificar que no haya errores de especificación en el software elegido. Constituye la base en las competencias estadísticas.
- Razonamiento estadístico: capacidad de seleccionar y justificar procedimientos, interpretar resultados más allá de la consigna y detectar errores. En la práctica, supone tareas como elegir la prueba estadística adecuada ante un problema dado, explicar por qué se rechaza o no una hipótesis, o detectar que un gráfico generado por software no es apropiado para los datos analizados.
- Pensamiento estadístico: comprensión del “para qué” y “en qué contexto”, formular problemas en términos estadísticos, reconocer la variabilidad como un aspecto inherente, evaluar supuestos, conectar datos con procesos generadores, tomar decisiones bajo incertidumbre, comunicar conclusiones con rigor disciplinar.

i Objetivo del curso (actividades prácticas)

El objetivo mínimo de este curso es que, al finalizar, cada estudiante demuestre al menos habilidades relacionadas con el **razonamiento estadístico** en problemas estructurados, y principios de **pensamiento estadístico** en problemas abiertos de su disciplina: no se

exige dominio pleno del pensamiento estadístico, pero sí que el estudiante sea capaz de identificar cuándo el razonamiento no es suficiente y qué preguntas de contexto quedan abiertas.

En virtud de lo anterior, ciertos contenidos usualmente incluidos en cursos introductorios de bioestadística (como algunos tipos de gráficos, ciertas distribuciones de probabilidad, etc.) han sido deliberadamente omitidos a fin de priorizar un tratamiento en mayor profundidad de aquellos conceptos y herramientas que se considera permiten desarrollar mejor los criterios de análisis, interpretación y toma de decisiones.

2.2. Evaluación práctica

La propuesta didáctica de este material se inspira en recomendaciones internacionales para la enseñanza de la estadística, como el estándar **GAISE**, adaptándolas al contexto de formación situada en la provincia de Jujuy.

i Metas del estándar GAISE

1. Enseñar pensamiento estadístico.
2. Enseñar la estadística como un proceso de investigación para la resolución de problemas y la toma de decisiones.
3. Brindar a los estudiantes experiencia en pensamiento multivariable.
4. Enfocarse en la comprensión conceptual.
5. Integrar datos reales con un contexto y un propósito.
6. Fomentar el aprendizaje activo.
7. Utilizar la tecnología para explorar conceptos y analizar datos.
8. Usar evaluaciones para mejorar y valorar el aprendizaje de los estudiantes.

Con ese marco, al finalizar el curso, **cada estudiante debe ser capaz de:**

- **Seleccionar y aplicar correctamente**, de forma autónoma, una técnica estadística ante un problema concreto.
- **Justificar su elección** (supuestos, limitaciones, alternativas).
- **Ejecutar correctamente** la técnica estadística seleccionada en al menos un entorno de software (acción y resultado).
- **Interpretar los resultados** tanto en términos estadísticos como disciplinares: agro-nomía, biología, ciencias ambientales, bromatología, etc.
- **Poder identificar al menos una fuente de variabilidad** no controlada en el problema.

- Señalar una limitación del análisis que requeriría datos adicionales, un ajuste del diseño o un análisis diferente.



Modalidad de las evaluaciones

- Los parciales serán presenciales y podrán ser realizados en grupos de hasta 2 (dos) personas. Deberán contar con una computadora ya que deberán realizar cálculos estadísticos (tomar las previsiones con tiempo, informar a la cátedra si se dificulta contar con una computadora para tratar de resolver la situación). En función de la cantidad de estudiantes, el formato de entrega podrá ser escrito o bien mediante la entrega del script con la resolución de los ejercicios. Además de la entrega, los parciales deberán ser defendidos/validados en forma oral, cuya implementación será oportunamente consensuada.
- Los informes de análisis y tareas de campo deberán ser defendidas en forma oral, cuya implementación será oportunamente consensuada.

2.3. Estrategia pedagógica

Se propone una integración gradual entre la práctica guiada y la construcción del razonamiento. Desde la primera clase se combinan:

- **Secuencias simples de manipulación y análisis de datos** con software estadístico especializado, como mecanismos de estímulo-respuesta (acción - resultado inmediato) que permiten al alumno asociar rápidamente una operación, como hacer click en un menú o ejecutar una línea de código, obteniendo una salida numérica o gráfica, generando así un primer anclaje sobre conceptos abstractos.
- **Ejercicios dirigidos** (para fijar alfabetización y primeros pasos del razonamiento).
- **Preguntas de reflexión crítica** (¿por qué un gráfico puede sugerir comportamientos atípicos? ¿qué pasaría si se modifican valores en el set de datos?).
- **Análisis y discusión de datos reales** (para desarrollar un sentido crítico y razonamiento estadístico).

Las clases prácticas se organizarán en dos etapas: la primera etapa, hasta el primer parcial, tendrá una orientación principalmente focalizada en lograr que los estudiantes adquieran destrezas procedimentales de análisis estadístico, tanto en el manejo conceptual de terminología estadística orientada a su

aplicación, como la manipulación del software estadístico. Es por ello que se aplicará una estrategia de aprendizaje basado en problemas (ejercicios). Una vez finalizada esta etapa, se asumirá que el estudiante ya cuenta con las competencias básicas para avanzar hacia el desarrollo del razonamiento y el pensamiento estadístico, momento en que se incorporará, además, un enfoque basado en proyectos.

En este marco, se reconoce una progresión en los niveles de aprendizaje:

1. Cuando el estudiante se centra en la sintaxis de un programa o en la identificación de procedimientos adecuados, se encuentra en una etapa de alfabetización estadística.
2. Cuando es capaz de justificar la elección de un método o modelo en función de los datos, avanza hacia el razonamiento estadístico.
3. Cuando cuestiona la adecuación de ese modelo considerando los mecanismos subyacentes del fenómeno en estudio, incorpora elementos del pensamiento estadístico.

Esta secuencia no debe interpretarse como etapas rígidas o lineales, sino como niveles que se superponen y retroalimentan a lo largo del proceso de aprendizaje.

2.4. Estrategias de estudio

Las siguientes estrategias están orientadas a fortalecer el razonamiento estadístico y mejorar la comunicación de resultados, priorizando la comprensión de los conceptos por sobre la aplicación mecánica de técnicas. Su objetivo es guiar el estudio y la preparación de evaluaciones, promoviendo una mirada integrada entre teoría, práctica e interpretación.

- Comprender antes de memorizar: identificar qué problema estadístico resuelve cada método.
- Practicar interpretación: explicar siempre qué significa un resultado en términos de un fenómeno real.
- Conectar conceptos: vincular muestreo, inferencia, modelos y diseño experimental.
- Preparar ejemplos: practicar la explicación con una o dos situaciones de ejemplo. Aplicar el lema: *“ejemplos, ejemplos y más ejemplos”*.
- Explicar supuestos: todo método estadístico tiene condiciones de validez.
- Pensar como investigador: preguntarse siempre cuál es la pregunta científica.
- Sostener el razonamiento: si no se recuerda una fórmula exacta, explicar el razonamiento estadístico.

Jerarquía para explicar un problema estadístico

Es importante poder explicar cualquier problema estadístico en el siguiente orden jerárquico:

1. **Pregunta de investigación** (o problema a resolver)
2. **Datos** (reconocer variables, estructura, procedencia, calidad, etc.)
3. **Descripción** (caracterizar estadísticamente los datos)
4. **Método a utilizar** (incluye especificar criterios de selección del método o modelo)
5. **Resultado**
6. **Interpretación**
7. **Comunicación**

2.5. Inteligencia artificial generativa: Pautas de trabajo en el curso

En 2024, la UNESCO (2024) publicó la *Guía para el uso de IA generativa en educación e investigación*. En ella, recomienda:

Recomendación UNESCO (2024)

“Evitar el uso de la IA generativa cuando pudiera privar a los estudiantes de oportunidades para desarrollar capacidades cognitivas a través del razonamiento lógico independiente.”

Esta recomendación guiará el enfoque pedagógico adoptado en el presente curso, orientando el uso de estas herramientas hacia un rol complementario y no sustitutivo del aprendizaje.

El desarrollo vertiginoso de la inteligencia artificial generativa (IAGEN o simplemente IA) supone la coexistencia de dos aspectos complementarios, inseparables, con implicancias profundas en el proceso de formación profesional:

1. **Oportunidad extraordinaria:** permite lograr una formación técnica y profesional de mayor profundidad, superando limitaciones de accesibilidad a la información y aspectos técnicos u operativos para focalizar en las bases científicas y éticas del conocimiento.
2. **Responsabilidad y desafío:** exige desarrollar un uso crítico, reflexivo y éticamente fundamentado, evitando la delegación acrítica del razonamiento. Se vuelve central fortalecer la capacidad de formular problemas, evaluar la pertinencia de métodos, validar resultados y sostener una interpretación fundada en criterios disciplinares.

2.5.1. ¿Qué puede aportar la IA en Bioestadística?

- Explicar qué hace una función, o interpretar un mensaje de error.
- Sugerir código o secuencias de análisis para resolver tareas analíticas.
- Reformular explicaciones estadísticas desde distintos enfoques.
- Proponer alternativas cuando un procedimiento no funciona.

2.5.2. Lo que la IA NO puede hacer

- **Comprender** el contexto específico del análisis, ni asumir las consecuencias de los resultados obtenidos.
- **Garantizar** que el código generado o la secuencia de análisis propuesta sean correctos para un determinado problema.
- **Sustituir el criterio** para elegir una medida o un método estadístico.
- **Interpretar resultados** con fundamento disciplinar (agronómico, bromatológico, biológico o ambiental).

2.5.3. Esquema de trabajo propuesto

Se propone el siguiente esquema de trabajo con herramientas de IA (como **NotebookLM** de Google u opciones similares):

1. **Crear un espacio de trabajo** (carpeta o notebook) y llamarlo con el nombre de la materia: Bioestadística.
2. **Cargar los materiales del curso:**
 - La guía: *Apuntes de Bioestadística con ejercicios resueltos*.
 - La bibliografía sugerida por la cátedra.
3. **Interactuar bajo el rol de asistente pedagógico:**
 - Resolver dudas conceptuales.
 - Guiar el aprendizaje mediante preguntas orientadoras.
 - Generar explicaciones alternativas (analogías, ejemplos).
 - Producir recursos complementarios (cuestionarios, esquemas, resúmenes, etc.).
 - Entender mensajes de error.
 - Comprender la sintaxis u opciones en diferentes programas estadísticos.
 - Explorar alternativas de análisis.

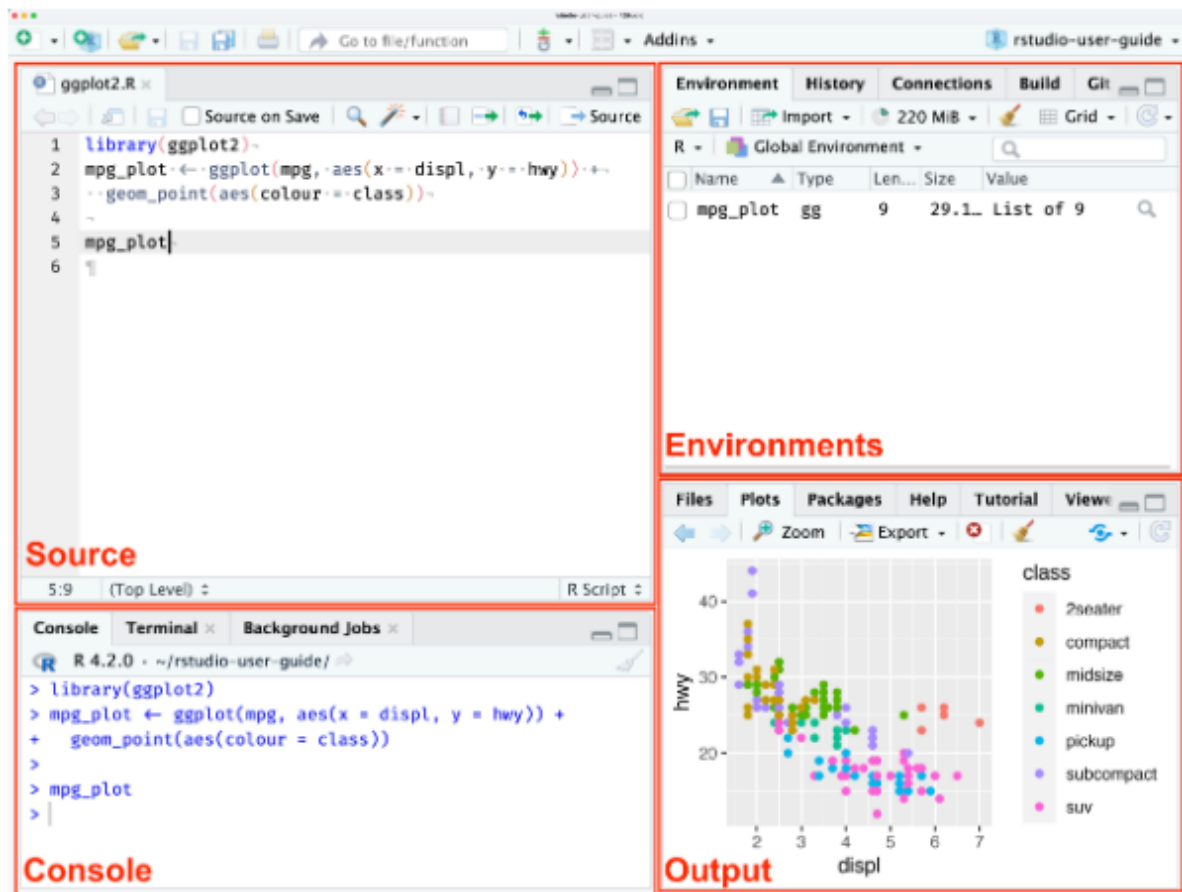
Criterios de precaución y verificación obligatoria

- Naturaleza Probabilística: Las herramientas de IA operan mediante modelos probabilísticos de lenguaje y no mediante algoritmos lógicos o fórmulas matemáticas cerradas. Por lo tanto, pueden generar “alucinaciones”, produciendo explicaciones que parecen técnicamente coherentes, pero que no siempre son coherentes con los fundamentos teóricos de la estadística.
- Prevalencia bibliográfica: En caso de discrepancia entre una explicación generada por la IA y los marcos conceptuales desarrollados en la bibliografía propuesta por la cátedra, **deben prevalecer siempre el contenido de la obra y la bibliografía académica.**
- Criterio disciplinar: El criterio del investigador es el único capaz de validar si una interpretación es agronómica, biológica, ambiental o bromatológicamente plausible, más allá de la corrección sintáctica del código sugerido.
- Verificación Obligatoria: Todo código generado (especialmente en R, Python o Julia) debe ser verificado. Pequeños errores en funciones (por ejemplo, confundir parámetros de varianza con desvío estándar) pueden invalidar completamente una prueba.

3 Introducción al manejo de R

3.1. Primeros pasos

- La instalación del lenguaje **R** puede realizarse desde la **Red Integral de Archivos de R (CRAN)**. En esta interface se debe seleccionar el sistema operativo que corresponda, por ejemplo **WINDOWS**. En la siguiente interface, seleccionar **base** (o bien “**install R for the first time**”) y, finalmente, en la siguiente pantalla, seleccionar Download R-4.4.2 for Windows (o la versión que se desee). Se descargará un archivo ejecutable con el instalador del lenguaje, el cual deberemos correr aceptando todas las instancias.
- **RStudio** es un *entorno de desarrollo integrado* (**IDE**, por sus siglas en inglés) para el lenguaje de programación R, que ofrece una interfaz gráfica de usuario (GUI) amigable y diversas herramientas para facilitar el desarrollo, depuración y visualización de código. Entre sus características se incluyen la edición de código con resaltado de sintaxis, la ejecución de código línea por línea o por bloques, la gestión de paquetes, la creación de gráficos y la integración con otros programas y herramientas para el análisis de datos.



- **RStudioCloud:** Actualmente administrada por la compañía *Posit*, permite trabajar con Rstudio en la nube. Es necesario crear un cuenta (“Sign up”) cuando se accede por primera vez, o bien ingresar los datos de acceso (“log in”) en caso de que ya se posea una cuenta. La versión gratuita permite actualmente trabajar hasta con 50 proyectos y 25 horas de trabajo mensuales.

Enlaces

- [CRAN\(“click”\)](#). Descarga del *lenguaje* R.
- [Rstudio\(“click”\)](#). Descarga del *IDE* RStudio.
- [Rstudio Cloud\(“click”\)](#).

i Compatibilidad

R 4.1 es la última versión compatible con versiones de 32 bits. Windows Vista ya no es compatible.

R 4.2.0 y versiones posteriores requieren Universal C Runtime (UCRT), que se incluye en Windows 10 y Windows Server 2016 o posterior. En versiones anteriores de Windows, UCRT debe instalarse antes de instalar R. UCRT está disponible para Windows desde Windows Vista SP2 y Windows Server 2008 SP2.

💡 Importante

Antes de instalar *RStudio*, tienes que haber instalado R.

3.1.1. Otros IDE

- [Navure](#) (“click”)
- [InfoStat](#) (“click”)
- [Jupyter](#) (“click”)
- [JASP](#) (“click”)
- [JAMOVİ](#) (“click”)
- [StatET](#) (“click”)
- [ESS](#) (“click”)
- [R Analytic flow](#) (“click”)
- [Rattle](#) (“click”)

💡 Tip

!!! R PUEDE SER EJECUTADO SIN NECESIDAD DE UN IDE!!!

Por ejemplo, en la barra de búsqueda de *Windows*, seleccionar **Editar las variables de entorno de esta cuenta**. Aparecerá la ventana **Variables de entorno**. En el bloque **Variables de usuario para usuario**, seleccionar **Path** y luego presionar **Editar...**. Aparecerá la ventana **Editar las variables de entorno**. Presionar la opción **Nuevo** e ingresar **C:\Program Files\R\R-4.4.2\bin** (o la versión que se desee). Finalmente, *Aceptar*.

Desde ahora podremos correr código R desde la ventana de comandos **Símbolo del sistema**, a la cual se accede ingresando el comando **CMD** en la barra de búsqueda de *Windows*. Basta ejecutar el comando **r** para comenzar a correr código de R, y **q()** para salir.

3.2. Objetos

“Todo en R es un objeto”

Se conoce como objeto a toda entidad creada y manipulada en **R**. Datos, tablas, gráficos, funciones, vectores, matrices, son algunos tipos de objetos en R.

Los objetos en R se caracterizan porque pueden presentar distintos atributos, por ejemplo, un nombre. Esto quiere decir que al momento de almacenar un objeto es posible asignarle un nombre.

i Nota

Al momento de nombrar un objeto en **R**, es importante utilizar nombres claros y cortos que permitan manipular con sencillez el objeto. También se sugiere utilizar nombres que permitan recordar fácilmente el contenido de los objetos.

Los nombres pueden incluir letras, números (siempre y cuando estén precedidos por una letra), guión bajo y puntos (antes o después de las letras). No se pueden utilizar símbolos especiales (“*”, “:”, “-”, “~”, “\$”, entre otros) ni palabras reservadas (NA, for, while, TRUE, FALSE, Inf, NaN, NULL) en la denominación de un objeto.

R distingue mayúsculas y minúsculas, por lo que las variables *a* y *A* serán consideradas como objetos diferentes.

Podemos almacenar los datos en variables por medio del operador “=” o bien por medio de los operadores de asignación “<-” o “->”. También podemos asignar valores utilizando la función `assign()`.

La función `class()` nos devuelve el atributo *tipo de dato*.

```
a = 1
a
class(a)

a <- 1
a
class(a)

1 -> a
a
class(a)

assign("a", 1)
a
class(a)
```

i Nota

Notar que las asignaciones se realizaron de derecha a izquierda en los dos primeros casos, y de izquierda a derecha en el tercero. La práctica más extendida es realizar la asignación de derecha a izquierda.

Ejemplos de atributos

- Class
- Dim: dimensión (matrices, dataframes, arreglos)
- Names: nombre de los componentes de un objeto
- Length: longitud de un vector.
- Ncol, Nrow: número de columnas/filas (en matrices, dataframes, etc.)
- Frequency: frecuencia en una serie de tiempo
- Etc.

i Tipos de datos

En R, los tipos de datos principales se pueden agrupar en **tipos atómicos**, que son las unidades mínimas de información, y las **estructuras de datos**. Los principales tipos de datos atómicos son:

- numeric: son datos numéricos. Un caso especial de datos numéricos es el tipo integer (entero).
- character: son datos de tipo texto
- logical: son datos lógicos, que pueden adoptar uno de dos valores: TRUE o FALSE.
- complex: Números complejos, por ejemplo $a = 2 + 3i$.

También puede mencionarse el tipo de dato raw, usado para almacenar secuencias de bytes útiles para trabajar con datos binarios, pero su empleo se realiza en contextos muy específicos, por lo que usualmente no trabajaremos con ellos.

Además, según la estructura de los datos, los objetos pueden ser:

- Vectores: son series de objetos del mismo tipo atómico.
- Matrices: son arreglos en filas y columnas de objetos del mismo tipo atómico.
- Arreglos (“arrays”): extienden las matrices a más dimensiones.

- Tablas(data.frames o tibbles): son arreglos en filas y columnas de objetos que pueden pertenecer a diferentes tipos atómicos.
- Listas: son series de objetos que pueden presentar diferentes estructuras.
- Factores: son series de objetos indexados o que presentan un ordenamiento jerárquico. Para indicar el índice de un objeto, se emplea `[[ ]]` en lugar de `[ ]`.

3.2.0.1. Ejemplos de clases de datos

```
x = 5
class(x)
mode(x)

x = 5L
class(x)

z = seq(0, 10, 2) # Secuencia: 'desde', 'hasta', 'cada'
class(z)

z = as.integer(z)
class(z)

sexo = c("masculino", "femenino")
class(sexo)

notas = c("1", "2", "3", "4", "5", "6", "7", "8", "9", "10")
class(notas)
mean(notas)
notas = as.numeric(notas)
class(notas)
mean(notas)

condicion = (z > x)
condicion
class(condicion)

complejo = 2 + 3i
complejo
class(complejo)
```

```

calificacion = factor(levels = c("insuficiente","regular","sobresaliente"))
calificacion
class(calificacion)

factor.sexo = as.factor(sexo)
factor.sexo
class(factor.sexo)

matrizNotas = matrix(notas, nrow = 5, ncol = 2)
matrizNotas
class(matrizNotas)

serie = seq(12)
serie
miArreglo = array(serie, dim = c(2,2,2)) # Las dimensiones hacen referencia al número de fil
miArreglo

miLista = list(sexo, notas, calificacion)
miLista

miLista1 = miLista[[1]]
miLista1
class(miLista1)

tabla = data.frame(
  Alumno = c("Juan", "Ana", "María"), # Columna 1
  Notas = c(5, 8, 10) # Columna 2
)
tabla

```

💡 Comentarios (#)

En **R**, toda línea precedida por el símbolo # (numeral) no es ejecutada. Su empleo es muy útil para realizar comentarios o descripciones dentro del código.

Los tipos de estructuras más utilizados en el análisis de datos estadísticos son los vectores, matrices, tablas y factores.

Podemos ver que es posible modificar la tipología de un dato por otra, de ser necesario. Algunas funciones utilizadas son:

- `as.numeric()`; `as.integer()`

- `as.character()`
- `as.factor()`
- `as.logical()`
- `as.data.frame()`
- `as.array()`

3.2.0.2. Borrado de objetos

Para borrar objetos almacenados, se utiliza la función `rm()`.

```
# Borrado de un objeto

rm(a)
a
# Error: objeto 'a' no encontrado

# Borrado de todos los objetos en el entorno

rm(list = ls())
```

Tip

Antes de comenzar cualquier análisis estadístico, se sugiere borrar todos los objetos que pudieran estar almacenados en el entorno.

3.3. Funciones

Todo lo que ocurre en R es el resultado de una función

Una función en **R** es un bloque de código que realiza una tarea específica. Se la invoca por medio de un nombre seguido de paréntesis que pueden contener argumentos si son necesarios. Por ejemplo:

```
a = 30
b = 20

sort(c(a, b))

sum(a, b) # equivalente a 'a + b'
```

R tiene muchas funciones integradas para realizar diversas operaciones, como cálculos matemáticos, manipulación de datos, gráficos, etc. Ejemplos incluyen **sum()**, **mean()**, **length()**, **plot()**, **sort()**, entre otros. Aún así, es posible crear funciones propias, lo cual se trabajará más adelante en la cursada.

3.3.1. Paquetes

En R, todas las funciones y sets de datos están almacenados en *paquetes*. Sólo cuando se *cargue* un paquete su contenido estará disponible. Ésto se realiza por una cuestión de eficiencia en el uso de los recursos computacionales, y también por seguridad.

Algunos paquetes o *librerías* forman parte del código fuente de R, y son conocidos como librerías de *base*, disponibles con la instalación de R. El resto de las librerías deben ser instaladas desde el *CRAN*, mediante el comando `install.packages("nombre del paquete")`. En este último caso, para poder utilizar las funciones de estas librerías, se debe correr el comando `library("nombre del paquete")`.

i Nota

También es posible instalar paquetes desde un archivo previamente descargado desde el CRAN. Para ello se puede correr el comando:

```
install.packages(file.choose(), repos = NULL, type="source")
```

Por ejemplo, para importar archivos de datos con distintas extensiones, se puede utilizar la función **import()** de la librería *rio*.

```
install.packages("rio") # instalación de la librería (solo una vez)
library("rio") # invocación de la librería (cada vez que deseemos utilizar alguna de sus funciones)
import(file.choose()) # función import para leer archivos de datos y file.choose() para seleccionar el archivo
```

Algunas de las librerías más utilizadas en R están “empaquetadas” en la librería *Tidyverse*.

3.3.2. Concatenación de funciones: el operador “|>” (*pipe*)

Con frecuencia es necesario “anidar” funciones en **R**, por ejemplo:

```
numeros = seq(20)
numeros
round(sqrt(mean(numeros)), digits = 2)
```

En la función anterior, la función **mean()** está anidada dentro de la función **sqrt()**, que a su vez está anidada dentro de la función **round()**. En estos casos, se dificulta llevar un control de las operaciones realizadas, por lo que se suele recurrir al operador |> (“pipe”), que permite concatenar funciones:

```
numeros = seq(20)
numeros |> mean() |> sqrt() |> round(digits = 2)
```

o bien:

```
numeros |>
  mean() |>
  sqrt() |>
  round(digits = 2)
```

3.4. Estructuras de datos

3.4.1. El vector

En **R**, un vector es una serie o conjunto de elementos del *mismo tipo*. Un vector puede definirse por medio del operador **c()**, y ser asignado a una variable por medio del operador **=** ó **<-**. Por ejemplo:

```
decimales = c(1,2,3.5,7.8,9); decimales; class(decimales)
enteros = c(1L, 2L, 3L, 7L, 9L); enteros; class(enteros)
# o bien
enteros = as.integer(decimales); enteros; class(enteros)
logicos = c(TRUE,FALSE,FALSE,TRUE); logicos; class(logicos)
```

Nota

Notar el empleo de “;” para concatenar diferentes comandos

En R, los vectores constituyen las estructuras de menor nivel. Es decir, en R, no existen las magnitudes *escalares*, las cuales son consideradas como *vectores de tamaño 1*.

Cuando en un vector hay elementos de distinto tipo, R automáticamente “coerciona” elementos de un tipo y los asigna a otro tipo en el siguiente orden: character, numeric, integer.

```
caracteres = c(1,2,"a","b",3); caracteres; class(caracteres)
sort(caracteres) #ordenamiento de caracteres

numeros = c(1.5, 2.5, 3L, 5L, 7.5); sort(numeros);class(numeros)
```

Operaciones aritméticas, relaciones e indexación

```
# Operaciones aritméticas

numeros = c(1:10)

sum(numeros)

mean(numeros)

a = 4

numeros + a

numeros - a

numeros * a

numeros / a

numeros %/% a

numeros %% a

numeros2 = c(11:20)

numeros + numeros2
```

```

b = c(3,4)

numeros + b # ojo!

a^2

a**2

# Relaciones (se aplica a condiciones)

1 < a

5 < a

b > a

b >= a

b == a

b != a

a %in% b

a %in% numeros

# Indexación

n2 = numeros[2]; n2

mean(numeros[2:10])

mean(numeros[2:length(numeros)])

mean(numeros[-1])

numeros[numeros >= 5]

mean(numeros[numeros < 8])

```

3.4.2. Matrices

Una matriz es un arreglo *bidimensional* de datos *del mismo tipo*. De hecho, un vector puede ser visto como un caso especial de una matriz en la cual hay solo una fila (o solo una columna). En el caso de las matrices, es posible consultar el atributo `dim()`:

```
serie = c(1:16); serie
matriz = matrix(serie,ncol = 4); matriz
matriz = matrix(serie,ncol = 4,byrow = TRUE); matriz
dim(matriz)

# indexación

matriz = matrix(serie,ncol = 4)
matriz[,2] # columna 2
matriz[2,] # fila 2
matriz[2,2] # columna 2 fila 2

# operaciones sobre filas y columnas

colSums(matriz)

colMeans(matriz)

rowSums(matriz)

rowMeans(matriz)

apply(matriz,1,sum) # suma marginal de filas
apply(matriz,2,sum) # suma marginal de columnas
```

3.4.3. Uniones

Son funciones que permiten combinar objetos como vectores, matrices y tablas.

3.4.3.1. Unión “cbind” (unión de “columnas”)

```
matriz.union.col = cbind(decimales,enteros); matriz.union.col
class(matriz.union.col)
dim(matriz.union.col)
```

3.4.3.2. Unión “rbind” (unión de “filas”)

```
matriz.union.fila = rbind(decimales,enteros); matriz.union.fila
class(matriz.union.fila)
dim(matriz.union.fila)
```

3.4.4. Data frames

A diferencia de una matriz, un *dataframe* puede ser visto como una colección de vectores de distinto tipo. Por ejemplo:

```
letras = letters[1:10]
numeros = c(1:10)
df = cbind(letras,numeros)
df
class(df)
```

Entonces:

```
letras = letters[1:10]
numeros = c(1:10)
df = data.frame(letras=letras,numeros=numeros)
df
class(df)
```

i Nota

Una *dataframe* puede ser visto como una unión de vectores:

```
colNumeros = df$numeros
colNumeros
summary(colNumeros)
```

Notar el empleo del signo \$, indicando que el vector “numeros” *pertenece* al objeto “df”.

Un objeto *tibble* es un data frame mejorado:

```
letras = letters[1:10]
numeros = c(1:10)
df = tibble(letras=letras,numeros=numeros)
df
class(df)
```

3.4.5. Otras estructuras

Como se mencionó, además de los vectores, matrices y data frames, otros tipos de estructura de datos son los arreglos o “arrays” y las listas, pero difícilmente las utilizaremos en durante la cursada.

3.5. Importación de datos

3.5.1. Archivos .xlsx

```
library(readxl)

datos = file.choose() |>
  read_excel()

View(datos)

rm(datos)
```

3.5.2. Archivos .csv

```
datos = read.csv(file.choose(), header = TRUE, dec = ",")

# 0 bien
```

```
datos = file.choose() |>
  read.csv(header = TRUE, dec = ",")

View(datos)
rm(datos)
```

3.5.3. Archivos en general (librería “{rio}”)

```
library(rio)

datos = file.choose() |>
  import()

View(datos)
rm(datos)
```

3.5.4. Desde el repositorio de R

R provee un extenso listado de sets de datos reales para entrenamiento, que puede consultarse por medio de la función **data()**. En caso de que se desee importar un set de datos en particular, se puede ejecutar la instrucción **data(*nombre del set de datos*)**, o simplemente, el nombre del set de datos. En nuestro caso:

```
datos = PlantGrowth
```

Para consultar mayores detalles acerca del set de datos, se puede ejecutar:

```
?PlantGrowth
```

3.6. Exploración de datos

```
str(datos) # estructura
head(datos,10) # primeros 10 registros
tail(datos,10) # últimos 10 registros
summary(datos) # resumen
```

3.7. Exportación de datos

Veremos únicamente cómo exportar un archivo en formato excel con el paquete {rio}:

```
install.packages("rio")

library(rio)

ruta = choose.dir() # selección destino (carpeta)

nombreArchivo = "crecimientoPlantas.xlsx"

pg |> # Exportación en formato .xlsx
  export(
    file = file.path(ruta, nombreArchivo)
  )
```

3.8. Funciones (Profundización, lectura optativa)

Como se estableció anteriormente, en **R**, las funciones son bloques de código reutilizables que realizan una tarea específica. Toman unos valores de entrada o **argumentos**, ejecutan una serie de operaciones y devuelven un resultado. Las funciones permiten organizar el código, evitar la repetición y facilitar el mantenimiento.

3.8.1. Estructura general de una función

```
nombre_funcion = function(argumento1, argumento2, ...) {
  # Cuerpo de la función
  # Operaciones con los argumentos
  return(resultado) # Devuelve un valor (opcional)
}
```

Componentes de una función:

- **Nombre de la función:** Identificador que se usa para llamar a la función.

- **Argumentos:** Valores de entrada que la función necesita para operar. Pueden ser opcionales o tener valores por defecto.
- **Cuerpo de la función:** Contiene las operaciones que se ejecutan cuando se llama a la función.
- **Valor de retorno:** El resultado que devuelve la función (usualmente usando la función `return()`). Si no se especifica, la función devuelve el valor de la última expresión evaluada.

Ventajas de utilizar funciones (Wickham, 2024)

- Se puede dar a la función un nombre evocador que hará el código más fácil de entender.
- A medida que cambien los requerimientos, solo se necesitará cambiar el código en un solo lugar, en vez de en varios lugares.
- Se eliminan las probabilidades de errores accidentales al copiar y pegar (por ej., al actualizar el nombre de una variable en un lugar, pero no en otro).

Ejemplo de una función básica

```

multiplicar = function(a, b = 2) {
  return(a * b)
}
multiplicar(5) # Usa b = 2 por defecto
multiplicar(5, 3)

```

Funciones anónimas

Son funciones sin nombre que se definen y usan directamente.

```

(function(x) x^2)(3) # ¿Qué valor devuelve?

```

Funciones como objetos

Las funciones pueden asignarse a variables, pasarse como argumentos a otras funciones o devolverse como resultados.

```

operacion = multiplicar
operacion(2, 3)

```

Nota

Las variables definidas dentro de una función son locales y no afectan el entorno global.

3.8.2. Estructuras de control

3.8.2.1. Estructuras de control en R: if y for

En R, las estructuras de control permiten manejar el flujo de ejecución del código. Dos de las más comunes son las estructuras condicionales (**if**) y los bucles (**for**). A continuación, se describe su uso y se proporcionan ejemplos básicos.

3.8.2.1.1. Estructura condicional if

La estructura **if** se utiliza para ejecutar un bloque de código si se cumple alguna condición específica. Su sintaxis básica es:

```
if (condición) {  
  # Bloque de código que se ejecuta si la condición es verdadera  
}
```

Si la condición es verdadera (**TRUE**), se ejecuta el bloque de código dentro del **if**. En caso contrario, no se ejecuta.

Ejemplo:

```
x = 10  
  
if (x > 5) {  
  print("x es mayor que 5")  
}
```

También es posible agregar una cláusula **else** para manejar el caso en que la condición no se cumpla:

```
if (x > 15) {  
  print("x es mayor que 15")  
} else {  
  print("x no es mayor que 15")  
}
```

3.8.2.1.2. Bucles for

El bucle **for** se utiliza para repetir un bloque de código un número específico de veces, iterando sobre una secuencia de valores. Su sintaxis básica es:

```
for (variable in secuencia) {
  # Bloque de código que se ejecuta en cada iteración
}
```

En cada iteración, la variable toma un valor de la secuencia, y el bloque de código se ejecuta con ese valor.

Ejemplo:

```
for (i in 1:5) {
  print(paste("Iteración:", i))
}
```

También es posible iterar sobre vectores o listas:

```
frutas = c("manzana", "banana", "naranja")

for (fruta in frutas) {
  print(fruta)
}
```

3.8.2.1.3. Combinación de if y for

Las estructuras if y for pueden combinarse para crear lógicas más complejas. Por ejemplo, se puede usar un if dentro de un bucle for para realizar acciones condicionales en cada iteración.

Ejemplo:

```
numeros = c(1, 3, 5, 7, 10)

for (num in numeros) {
  if (num %% 2 == 0) {
    print(paste(num, "es par"))
  } else {
    print(paste(num, "es impar"))
  }
}
```

```
[1] "1 es impar"
[1] "3 es impar"
[1] "5 es impar"
[1] "7 es impar"
[1] "10 es par"
```

3.9. Micro-ejercicios de aplicación (optativos)

Los siguientes micro-ejercicios que te permitirán practicar los conceptos revisados en esta unidad (objetos, vectores, matrices, data frames, importación de datos, funciones y estructuras de control). Puedes copiar y pegar las preguntas en un script de R local, o bien utilizar **webR**, que es la versión *online* de R, disponible en esta guía.

```
# =====
# MICRO-EJERCICIOS DE APLICACIÓN - Introducción a R
# Ejemplos biológicos y ambientales del NOA
# =====

# -----
# I. OBJETOS, ASIGNACIÓN Y CLASES
# -----

# 1) Asigne a un objeto llamado 'altitud_pozuelos' el valor 3650 (m s.n.m.,
#     altitud aproximada de la Laguna de Pozuelos) usando el operador "<=".

# 2) Consulte la clase del objeto 'altitud_pozuelos' con class().

# 3) Asigne el nombre de la especie "Vultur gryphus" (cóndor andino) al objeto
#     'especie_condor' usando el operador "=". Verifique su clase.

# 4) Cree el objeto 'vicuna_avistada' con el valor lógico TRUE, indicando que
#     se registró presencia de vicuña (Vicugna vicugna) en un transecto de la Puna.
#     Verifique su clase.

# 5) Asigne el valor 4 (número de parches de bofedal muestreados) al objeto
#     'n_bofedales' utilizando la función assign().

# 6) Cree el objeto 'temp_media_puna' = 8.5 (°C) usando el operador "->".

# 7) Convierta el objeto 'n_bofedales' a tipo entero con as.integer() y
#     verifique su clase con class().
```

```

# 8) Borre el objeto 'temp_media_puna' del entorno con rm() y confirme el
#     error al intentar invocarlo nuevamente.

# -----
# II. VECTORES: CREACIÓN, OPERACIONES ARITMÉTICAS Y RELACIONES
# -----

# 9) Cree un vector 'peso_zorro' con las masas corporales (kg) de 5 ejemplares
#     de zorro colorado (Lycalopex culpaeus) capturados en la Quebrada de
#     Humahuaca: 4.8, 5.2, 4.5, 5.6, 4.9.

# 10) Calcule la media y el desvío estándar de 'peso_zorro'.

# 11) Cree el vector 'temp_max_yungas' con las temperaturas máximas diarias (°C)
#     registradas en el Parque Nacional Calilegua durante una semana:
#     28, 30, 27, 31, 29, 26, 30.

# 12) Calcule cuántos días la temperatura máxima superó los 29°C, usando una
#     operación de relación sobre 'temp_max_yungas'.

# 13) Cree el vector 'cobertura_cardon' con porcentajes de cobertura de cardón
#     (Echinopsis atacamensis) en 6 parcelas de la Quebrada: 12, 8, 15, 20, 10, 18.

# 14) Sume 5 puntos porcentuales a cada valor de 'cobertura_cardon' (simulando
#     un año de crecimiento) y guarde el resultado en 'cobertura_cardon_2027'.

# 15) Cree el vector 'ph_suelo_chaco' con valores de pH de suelo en 5 sitios del
#     Chaco jujeño: 6.8, 7.1, 6.5, 7.4, 6.9. Determine cuántos sitios son
#     alcalinos (pH > 7).

# 16) Cree el vector 'altura_algarrobo' con la altura (m) de 8 ejemplares de
#     algarrobo negro (Prosopis nigra): 6.2, 7.5, 5.8, 8.1, 6.9, 7.0, 5.5, 8.4.
#     Calcule el rango (máximo - mínimo) sin usar la función range().

```

```

# 17) Cree dos vectores, 'machos' y 'hembras', con el número de vicuñas
#      contadas por punto de muestreo (5 puntos cada uno):
#      machos = c(3,5,2,4,6); hembras = c(4,6,3,5,7).
#      Calcule el vector 'total_grupo' sumando ambos vectores elemento a elemento.

# 18) A partir de 'total_grupo', determine en qué puntos de muestreo el grupo
#      total superó los 9 individuos (operación de relación).

# 19) Cree el vector 'especies_yungas' con los nombres: "Tremarctos ornatus",
#      "Puma concolor", "Tapirus terrestris". Verifique su clase y longitud.

# 20) Convierta 'especies_yungas' a tipo factor con factor() y aplique
#      summary() para contar cuántos registros hay por especie.

# -----
# III. INDEXACIÓN DE VECTORES
# -----

# 21) A partir de 'peso_zorro' (ejercicio 9), extraiga el peso del tercer
#      individuo capturado.

# 22) A partir de 'temp_max_yungas' (ejercicio 11), extraiga las temperaturas
#      registradas entre el segundo y el quinto día.

# 23) A partir de 'cobertura_cardon' (ejercicio 13), excluya el primer valor y
#      calcule la media de los restantes.

# 24) A partir de 'altura_algarrobo' (ejercicio 16), extraiga únicamente los
#      valores mayores o iguales a 7 metros.

# 25) A partir de 'ph_suelo_chaco' (ejercicio 15), calcule la media de los
#      sitios con pH menor a 7.

```

```

# 26) A partir de 'total_grupo' (ejercicio 17), extraiga el valor máximo
#       registrado utilizando indexación combinada con la función which.max().

# -----
# IV. MATRICES
# -----

# 27) Un investigador registró la abundancia de cuatro especies de aves
#       acuáticas (filas) en tres lagunas altoandinas (columnas):
#       12,8,15, 5,10,7, 20,18,22, 3,6,4 (por filas).
#       Cree la matriz 'abundancia_aves' de 4 filas y 3 columnas (byrow = TRUE).

# 28) Consulte las dimensiones de 'abundancia_aves' con dim().

# 29) Extraiga la columna correspondiente a la segunda laguna muestreada.

# 30) Calcule la abundancia total registrada por laguna (suma por columnas).

# 31) Calcule la abundancia promedio por especie (promedio por filas).

# 32) Determine, mediante apply(), la laguna con mayor abundancia total de
#       aves acuáticas.

# -----
# V. UNIONES (cbind / rbind)
# -----

# 33) Cree dos vectores: 'sitio' = c("Yungas","Puna","Chaco") y
#       'riqueza_sp' = c(45, 18, 27) (riqueza de especies de aves por
#       ecorregión). Únalos por columnas con cbind() en el objeto 'union_col'.

# 34) Verifique la clase y dimensión de 'union_col'.

# 35) Una los mismos vectores por filas con rbind() y compare la estructura
#       resultante con la del ejercicio 33.

```

```

# -----
# VI. DATA FRAMES
# -----

# 36) Cree un data frame 'aves_NOA' con las columnas 'ecorregion' =
#      c("Yungas","Puna","Chaco") y 'riqueza' = c(45, 18, 27).

# 37) Verifique la clase de 'aves_NOA' y su estructura con str().

# 38) Extraiga la columna 'riqueza' utilizando el operador "$" y calcule su
#      media.

# 39) Agregue al data frame 'aves_NOA' una nueva columna 'superficie_km2' =
#      c(2500, 8600, 4100), correspondiente a la superficie aproximada
#      relevada en cada ecorregión.

# 40) Calcule la densidad de riqueza (riqueza / superficie_km2) para cada
#      ecorregión y almacénela en una nueva columna 'densidad_riqueza'.

# 41) Utilizando indexación de data frame ([fila, columna]), extraiga el
#      valor de riqueza correspondiente a la Puna.

# 42) Aplique summary() sobre el data frame 'aves_NOA' completo e identifique
#      qué ecorregión presenta mayor riqueza de aves.

# -----
# VII. IMPORTACIÓN Y EXPORTACIÓN DE DATOS ({rio})
# -----

# 43) Exporte el data frame 'aves_NOA' a un archivo "aves_NOA.xlsx" utilizando
#      la función export() de la librería {rio}.

# 44) Importe nuevamente el archivo "aves_NOA.xlsx" y asígnelo al objeto
#      'aves_importado'.

```

```

# 45) Verifique con class() e str() que la estructura de 'aves_importado'
#     coincide con la del objeto original 'aves_NOA'.

# 46) Exporte únicamente la columna 'riqueza' de 'aves_NOA' a un archivo CSV
#     llamado "riqueza_aves.csv".

# -----
# VIII. FUNCIONES PROPIAS Y ESTRUCTURAS DE CONTROL
# -----

# 47) Escriba una función 'convertir_mm_cm' que reciba un valor en milímetros
#     y devuelva su equivalente en centímetros. Aplíquela al vector 'svl'
#     (longitud hocico-cloaca) del Ejemplo 1 del script de clase.

# 48) Escriba una función 'clasificar_cobertura' que reciba un valor de
#     cobertura vegetal (%) y devuelva "Degradado" si es menor a 30, o
#     "Conservado" en caso contrario, utilizando una estructura if/else.

# 49) Utilizando un bucle for, recorra el vector 'cobertura_cardon' (ejercicio
#     13) e imprima, para cada parcela, si su cobertura es "Baja" (<15) o
#     "Alta" (>=15), combinando for e if.

# 50) Utilizando un bucle for sobre el vector 'peso_zorro' (ejercicio 9),
#     imprima un mensaje indicando el número de individuo y su peso en
#     gramos (convirtiendo de kg a g dentro del bucle).

```

4 Algunas herramientas

R en la web

En este entorno podrás ejecutar algunas funciones de R.

```
# Códigos acá
```

4.1. GeoGebra (Visualización interactiva)

Como complemento al uso de software estadístico especializado, se recomienda el empleo del módulo de **Distribuciones Estadísticas de GeoGebra**. Esta herramienta permite al estudiante seleccionar una distribución y especificar sus parámetros para obtener de forma inmediata la representación gráfica de la función de densidad (en variables continuas) o de la función de probabilidad (en variables discretas).

El valor didáctico de GeoGebra yace en su capacidad para calcular la probabilidad en un intervalo de forma visual, lo cual coincide con el área bajo la función de densidad o la suma de probabilidades discretas.

Integrar esta práctica ofrece las siguientes **ventajas estratégicas**:

- Andamiaje visual: Antes de realizar un cálculo con un programa estadístico, GeoGebra permite construir un modelo mental de la distribución. Esto ayuda a internalizar cómo los cambios en parámetros alteran la forma y posición de la curva.
- Validación de resultados: Funciona como un mecanismo de control para verificar si los valores de probabilidad obtenidos mediante software son consistentes con la representación geométrica.

- Transición del razonamiento al pensamiento: Al facilitar la manipulación interactiva de los límites de un intervalo, cada estudiante puede explorar escenarios del tipo “¿qué pasaría si...?”, fortaleciendo su capacidad de pensamiento estadístico al conectar la teoría abstracta con una representación espacial concreta.

La utilización del simulador de GeoGebra es muy intuitiva y de fácil apropiación por parte de los estudiantes. No obstante, durante los encuentros presenciales se realizarán instancias de acompañamiento y demostraciones guiadas orientadas a explicitar su funcionamiento y vincular las acciones realizadas en el entorno interactivo con los conceptos estadísticos subyacentes.

Referencias

- **Apuntes de Bioestadística. Con ejercicios resueltos.** Guía práctica de cátedra.
- Lepš, J., & Šmilauer, P. (2020). Biostatistics with R: An introductory guide for field biologists. Cambridge University Press.
- Balzarini, M., Di Rienzo, J., Tablada, M., Gonzalez, L., Bruno, C., Córdoba, M., Robledo, W., & Casanoves, F. (s.f.). Bioestadística: Primera edición. [Editorial desconocida].
- Weaver, K. F., Morales, V. C., Dunn, S. L., Godde, K., & Weaver, P. F. (2018). An introduction to statistical analysis in research: With applications in the biological and life sciences. John Wiley & Sons.
- Irizarry, R. A., & Love, M. I. (2021, March 17). Data analysis for the life sciences (Version 2021-03-17). Leanpub.
- Curry, E. (2020). Introduction to bioinformatics with R: A practical guide for biologists. CRC Press, Taylor & Francis Group.